Network Layer: Control Plane Formula Sheet (Ch. 5)

1. The Link-State (LS) Routing Algorithm

Dijkstra's Algorithm

An algorithm run by each router to compute the least-cost paths from itself (as the source node, 'u') to all other nodes in the network.

Notation

$c(x, y)$**:** Cost of the direct link between nodes x and y; ∞ if not direct neighbors.
$D(v)$**:** Current estimate of the cost of the least-cost path from source 'u' to destination 'v'.
$p(v)$**:** Predecessor node to 'v' on the current best path from 'u'.
N'**:** The set of nodes whose least-cost path from 'u' is definitively known.

The Algorithm

1: **Initialization:**
2: $N' = \{u\}$
3: **for all** nodes v **do**
4: **if** v is adjacent to u **then**
5: $D(v) = c(u, v)$
6: **else**
7: $D(v) = \infty$
8: **end if**
9: **end for**
10:
11: **Loop**
12: Find node w not in N' such that $D(w)$ is a minimum.
13: Add w to N'.
14: **for all** neighbors v of w that are not in N' **do**
15: $D(v) = \min(D(v), D(w) + c(w, v))$
16: **end for**
17: **until** all nodes are in N'.

The core update step, line 14, is the heart of the algorithm. It checks if the path to a neighbor 'v' can be improved by going through the newly added node 'w'.

Algorithm Complexity

Computational Complexity: With n nodes, a simple implementation has complexity $O(n^2)$. More efficient implementations using heaps achieve $O(n \log n)$.
Message Complexity: Each router must broadcast its link state to all other routers, which can be message-intensive.

2. The Distance-Vector (DV) Routing Algorithm

A decentralized, iterative algorithm based on the Bellman-Ford equation.

The Bellman-Ford Equation

The core of the DV algorithm. It states that the least cost from node 'x' to node 'y' is the minimum of the costs to get there through each of 'x''s neighbors 'v'.

$$D_x(y) = \min_v \{c(x, v) + D_v(y)\}$$

$D_x(y)$**:** The estimated cost of the least-cost path from 'x' to 'y'.

$c(x, v)$**:** The direct cost of the link from 'x' to its neighbor 'v'.
$D_v(y)$**:** The neighbor 'v''s estimated cost to get to 'y' (which 'v' has advertised to 'x').
$\min_v$**:** This minimization is performed over all of 'x''s direct neighbors.

The Algorithm (Conceptual)

Each node 'x' in the network executes the following loop:

1: **wait** for a change in local link cost or a message from a neighbor 'v'.
2: **recompute** its distance vector estimates using the Bellman-Ford equation for all destinations $y \in N$:
3: $D_x(y) \leftarrow \min_v\{c(x, v) + D_v(y)\}$
4: **if** node 'x''s own distance vector has changed **then**
5: **notify** all neighbors of the change by sending them the new distance vector.
6: **end if**

3. BGP (Border Gateway Protocol) Route Selection

When a BGP router learns multiple paths to the same destination prefix, it uses a hierarchical set of rules to select the single best path to install in its forwarding table. The selection stops at the first rule that breaks the tie.

1. **Highest Local Preference Value:** Choose the path with the highest 'local-pref' attribute. This is a purely policy-based decision set by the network administrator.
2. **Shortest AS-PATH:** Choose the path that traverses the fewest number of Autonomous Systems.
3. **Closest NEXT-HOP Router (Hot Potato Routing):** If AS-PATH lengths are equal, choose the path whose 'NEXT-HOP' router has the lowest intra-domain path cost (according to OSPF). This gets the traffic out of the local AS as quickly as possible.
4. **Additional Criteria:** If a tie still exists, other attributes (like MED - Multi-Exit Discriminator) and router IDs are used as tie-breakers.